

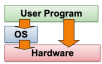
CS2106 Cheatsheet, AY26/27 S1

Zhao Wu and Tien Cheng

October 03, 2026

1 Introduction

OS is an intermediary program between computer hardware and user.

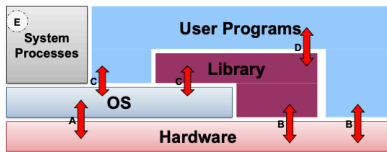
First Computer	OS Type: No OS - Program directly by physically changing the hardware configuration (cables, switches, punched paper tape) Advantage: - Minimal OS overhead Disadvantage: - Not portable (to port, need to manually rewrite the program) and inefficient - E.g. Electronic Numerical Integrator And Computer (ENIAC), Harvard Mark I	
Mainframes	OS Type: Batch OS - Execute one job at a time. - Support batch processing only - No interactive interface (program via paper tape, magnetic tape, punch card) - User interacts with hardware directly with additional information for OS (e.g. resource required, job specification) Disadvantage: - CPU is idle during I/O from simple batch processing - E.g. IBM 360	
Time-Sharing OS	OS Type: Time-Sharing OS Advantage: - Allow multiple users to interact with the machine using terminals (teletypes) - User job scheduling (illusion of concurrency) - OS manages CPU time, memory and storage - Virtualization of hardware (each program executes as if it has all the resources to itself) - E.g. Apple II PC, IBM PC	
Personal OS	Machine may be dedicated to user, not timeshared between users Windows model: - Single user at a time, but possibly more than 1 user can access Unix model (General time-sharing model): - One user at the workstation, but other users can access remotely	

Motivation of OS

- Abstraction:** Hide low-level details and present higher-level functionality to the user
- Resource Allocator:** Manage all resources (CPU, Memory, I/O) and arbitrate potentially conflicting requests for efficient and fair resource use
- Control Program:** Control execution of programs to prevent errors and improper use of the computer

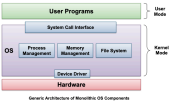
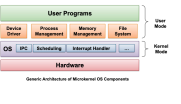
1.1 OS Structures

- OS is software that runs in **Kernel Mode** with direct access to all hardware resources (w/o system calls, normal libraries and 'normal' I/O in kernel mode)
- Other software operates in **User Mode** (i.e. limited/controlled access to hardware resources)



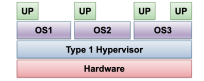
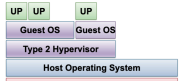
- A:** OS executing machine instructions
- B:** Normal machine instructions executed (program/library code)
- C:** Calling OS using **system call interface**
- D:** User program calls library code
- E:** System processes
 - Provide high-level services, usually part of OS

OS Structures

Monolithic	- Big program (e.g. Linux source code) - If Kernel fails, BSOD - Better performance	
Microkernel	- Smaller and cleaner abstraction - Provides basic and essential facilities: - Inter-Process Communication - Address space management - Thread management - Higher-level OS services are run outside of the kernel, using IPC to communicate. - Lower performance	

1.2 Virtual Machines

A **Virtual Machine** emulates the underlying hardware for running multiple OS, created and managed by a **Hypervisor / Virtual Machine Monitor (VMM)**

Type 1 Hypervisor	Type 2 Hypervisor
	
- Type 1 hypervisor: - AKA bare-metal hypervisor - Provides individual virtual machines to guest OSes - E.g., IBM VM/370, VMware ESXi	- Type 2 hypervisor: - Runs in host OS - Guest OS runs inside Virtual Machine - E.g., VMware Workstation, VirtualBox, GEMU

2 Process Abstraction

- Process** is the OS abstraction for a running program.

Process Context

Layer	Component	Stores
Memory	Text	program instructions
	Data	global variables and static variables
	Stack	collection of stack frames
	Heap	region of memory used to store dynamically allocated data
Hardware	General-Purpose Registers (GPRs)	temporary data
	Program Counter (PC)	next instruction address

Layer	Component	Stores
	Stack Pointer (SP)	top of stack frame address
	Frame Pointer (FP)	fixed location in stack frame
OS	Process ID (PID)	unique process ID
	Process State	current process state

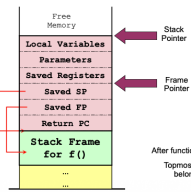
2.1 Memory & Function Calls

- Each function invocation **pushes a new stack frame onto the stack**, while each function return **pops the top stack frame**.
- FP** points to the fixed base of the current stack frame, allowing access of arguments and locals with constant offset.

Stack-Frame Component	Who	Why
Local variables	Callee	Callee's own local variables
Parameters (only those that don't fit in registers)	Caller	so the callee can read the parameters of function call
Saved GPRs	Callee	Copy of GPRs which callee modifies; protect the caller's values across the call
Saved old SP	Callee	restore the caller's SP
Saved old FP	Callee	restore the caller's FP
Return address / Saved PC	Caller	return to the right instruction after the callee returns

Function Call Convention

- Function Call Preparation:**
 - Caller: pass **parameters using registers and/or the stack**
 - Caller: save **return PC on stack**
- Transfer of Control from Caller to Callee:**
 - Callee: save registers used by the callee, old SP and FP
 - Callee: allocate space for **local variables of the callee on the stack**
 - Callee: adjust the SP (and FP) to **point to the new stack top**
- Callee Function Executes**
- Callee Function Returns**
 - Callee: save **return result in return register** (if applicable)
 - Callee: **restore saved registers and SP, FP**
- Transfer of Control from Callee back to Caller using saved PC:**
 - Caller: Use return result and continues execution of the program



Register Saving and Spilling

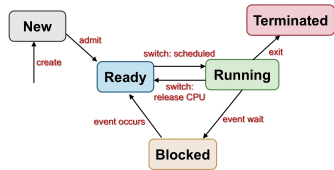
- Register Saving:** caller and callee may use the same registers, so we need to save the registers and restore them after the call.
- Register Spilling:** When a function has more arguments than the number of registers, the extra arguments are spilled to the stack
 - Callee-saved** (default 2106 convention): the callee saves the old values of the registers into its stack frame and restores them after the call
 - Caller-saved:** the caller saves needed registers into its stack frame and restores them after the call

Heap Memory

- heap** is a memory region to store dynamically allocated memory. (e.g. malloc and free in C)
- Dynamically allocated data **cannot be stored in:**
 - Data region:** size must be known at compile time
 - Stack:** the data may outlast the function call

2.2 Process States

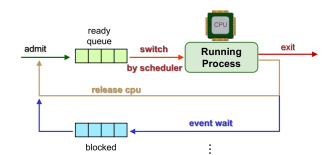
Process State Model



Multi-Process Management

- With 1 CPU core, at most 1 process can be running at a time (2106 convention)
- With m CPU cores, at most m processes can be running at a time

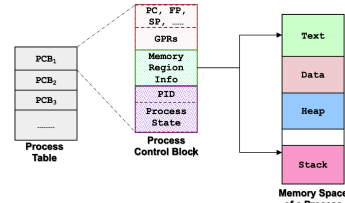
Process Queues



- The OS maintains a queue of processes for each state
 - Ready Queue:** processes that are ready to be scheduled to run on the CPU
 - Blocked Queue:** processes that are waiting for an event to occur
 - may be separate queues for diff types of blocked (e.g. I/O / signal blocked)

2.3 Process Control Block (PCB)

A **Process Control Block (PCB)** or **Process Table Entry** is a data structure that describes the execution context for a process, maintained by kernel.



2.4 System Calls

System Call is an OS API for a **user program** to request services in Kernel.

System Call Mechanism

- User invokes the library call (e.g. getpid())
- Library call places the system call number into a register.
- Library call invokes the **trap instruction to switch from user mode to kernel mode**.
- In kernel mode, the **dispatcher identifies the system call and passes control to the appropriate system call handler**.
- System call handler executes the system call
- System call handler ended by restoring CPU state and return to user mode.
- Library call returns the result to the user program.

Exception and Interrupt

- Exception:** Occurs due to program execution, synchronous
 - E.g. divide by zero error or a page fault
- Interrupt:** Occurs independent of program execution, asynchronous
 - E.g. hardware interrupt or a software interrupt

2.5 A Case Study of Processes in Unix

2.5.1 Process Creation: Fork -> Exec

fork()

int fork() creates a child process by duplicating currently executing parent.

- Both parent and child continue from **immediately after** the fork() call.
- Child is initially an **almost exact duplicate**:
 - Same code and **initial** address-space **contents** (conceptually duplicated, **not** shared)
 - Copied register and execution context
 - Different PID and PPID
- Different fork() return value: parent receives the child's PID; child receives 0

Nondeterministic scheduling

- Parent/child order is **nondeterministic**: parent first, child first, or interleaved.

Independent Address Spaces

- Parent and child initially **see equivalent values** after fork(), but do **not** share memory.
- Stack, heap, data, and code image are **conceptually duplicated**.
 - Modifying a variable in one process does **not** modify the other.
 - Shared kernel resources are inherited (e.g. opened files), not the address space.

exec() replaces the process image

exec() **replaces** the current process (code and data) with a new executable:

- Begins at the new program's entry point
- Discards the old stack and execution state
- Retains the same PID** and broader process identity
- Successful exec() does not return** — the old image no longer exists
- Variants: exec(full path, arg0, ..., NULL), execv("/bin/ls", args), execve, execlp(relative path, arg0, ..., NULL), execvp

main Command-Line Arguments

- int main(int argc, char **argv[], char **vp)**
- argc: number of arguments, **including** the program name
 - argv: array of C strings arguments; argv[0] is conventionally the executable name
 - vp: array of C strings environment variables

init and Process Tree

A process can only be created by forking an existing process, so Unix processes form a **process tree**.

- Root is **init**: created by the kernel during boot, traditionally **PID 1**
- Common ancestor of user processes; typically spawns OS/system programs
- Adopts orphaned processes
- Cannot be killed** even though it runs in user space; if it crashes → **kernel panic**

2.5.2 Process Termination & Lifecycle

exit()

exit(int status) terminates the current process. **Does not return**.

- exit status**: zero if normal/successful termination, non-zero if error/abnormal
- Returning from main() **implicitly** invokes exit() with return value becomes the exit status
- On process exit():
 - Process state is set to **Zombie**
 - Most resources are released (e.g. file descriptors)
 - Some information is **not releasable** (so the parent can wait()):
 - PID
 - Exit status
 - Process accounting (e.g. CPU time)

wait()

pid_t wait(int *status) lets a parent synchronise with any child(s):

- Blocks** until at least one child terminates
- Returns the PID of a terminated child
- Stores the child's exit status through status, else NULL
- Kernel can write into the parent's memory because it is privileged
- Variants:
 - waitpid() — wait for a specific child
 - waitid() — wait for child state changes

Zombie vs Orphan

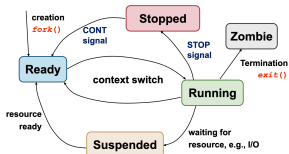
- Zombie**: child that has **exited** but **has not yet been consumed** by parent via wait()

- Occupies a process-table entry; too many can exhaust the table (older Unix: may need reboot)
- Orphan**: a **still-running** child whose **parent has terminated**
- init becomes its pseudo-parent
- When the child later terminates, init will clean up with wait()
- If the parent dies while the child is **already a zombie**, init reaps that leftover state too

Parent-Child Lifecycle

- Parent forks a child
- Child optionally execs a new program
- Child exits and becomes a zombie
- Parent waits
- Kernel removes the child's remaining process-table entry

Unix Process States



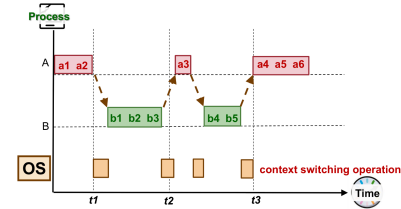
3 Process Scheduling

Concurrent processes refers to processes that progress in execution at the same time. This can be achieved by:

- Virtual Parallelism: Pseudo-parallelism
- Physical Parallelism: Multiple CPU / Core to allow parallel execution

Timeslicing

- Interleaving of instruction between processes is called **timeslicing**
- Context switching operation is required before one process can handover to another process.
 - Can be done in 1-Core CPU (timesliced for instructions) or multiprocessor (timesliced for instructions & CPU core)



3.1 Scheduling Algorithms

Scheduling Problem

When there are more ready-to-run processes than available CPUs, **Scheduler** decides process should be chosen to run based on **Scheduling Algorithm**

Scheduling Algorithm is tailored based on process behaviour and process environment.

- Process Behavior**: a typical process cycles between
 - CPU-activity: Compute-bound spends majority of time
 - IO-activity: IO-bound spends majority of time
- Processing Environment**:
 - Batch Processing: No interaction required, no need to be responsive.
 - Interactive (or Multiprogramming): Active user interacts with system. Should be responsive.
 - Real time processing: Have deadline to meet, usually periodic process.

For all processing environments:

- Fairness**: Processes should get a fair share of CPU time and no starvation
- Utilization**: All parts of computing system should be utilized

Scheduling Algorithms:

- Non-preemptive (Cooperative)**: Process stayed scheduled (in running state) until it blocks or give up the CPU voluntarily.
- Preemptive**: Process is given a fixed time quota to run, and at the end of the time quota, the process is suspended.

3.1.1 Scheduling for Batch Processing

Batched processing system has no user interaction and mostly dominated by non-preemptive scheduling.

Key criterion:

- Turnaround time: Total time taken (finish time - arrival time)
- Throughput: number of tasks finished per unit time
- CPU Utilization: Percentage of time when CPU is working on a task

First Come First Serve (FCFS)

Mechanisms:

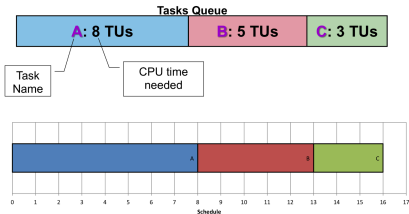
- Tasks are stored on First-In-First-Out (FIFO) queue.
- First task in the queue to run until task is done or task is blocked.
- Blocked task is removed from the FIFO queue, and placed at the back of queue when it is ready.

Characteristics:

- Non-preemptive algorithms
- No Starvation

Shortcomings:

- Convoy Effect: CPU-bound and IO-bound processes cannot be run concurrently.
- Reordering of task can reduce average waiting time.



- The average total waiting time for 3 tasks

$(0 + 8 + 13)/3 = 7$ Time Units

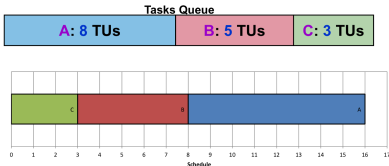
Shortest Job First (SJF)

Mechanism:

- Select the task with the smallest total CPU time

Characteristics:

- Non-preemptive algorithm
- Need to know total CPU time for a task in advance
- Starvation is possible as algorithm is biased towards short jobs
- Guarantees smallest average waiting time as task with shortest job always comes first



- The average total waiting time for 3 tasks

$(0 + 3 + 8)/3 = 3.66$ Time Units

Predicting CPU Time

- Guess the future CPU time requirement by previous CPU-Bound phases

$$\text{Predicted}_{n+1} = \alpha \text{Actual}_n + (1 - \alpha) \text{Predicted}_n$$

- Actual_n : Most recent CPU time consumed
- Predicted_n : Past history of CPU time consumed

- α : Weight placed on recent event
- Predicted_{n+1} : Latest prediction

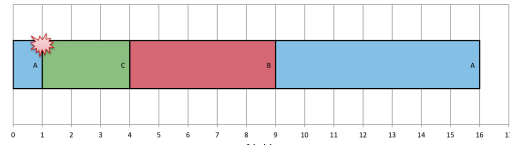
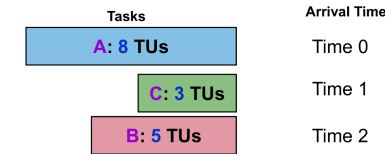
Shortest Remaining Time (SRT)

Mechanism:

- Select job with shortest remaining (or expected) time

Characteristics:

- Preemptive algorithm
- When a new shorter job arrives, will stop current job and switch to shorter job



3.1.2 Scheduling for Interactive Environment

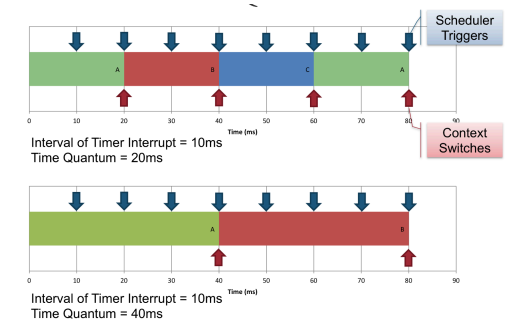
Interactive Environment uses mostly preemptive algorithm for good response time with scheduler that runs periodically.

Key Criterion:

- Response time
- Predictability

Timer & Time Quantum

- Interval of Timer Interrupt (ITI)**: time it takes to interrupt and invoking the OS scheduler, typically 1ms-10ms.
- Time Quantum**: Execution duration given to a process, that must be multiples of timer interrupt. Time Quantum could be constant or variable among the processes, typically 5ms-100ms.



Round Robin (RR)

Mechanism:

- Tasks are stored on First-In-First-Out (FIFO) queue.
- First task in the queue to run until a fixed time quantum elapsed, or task is done, or task is blocked.
- Task is then placed at end of queue for another turn
- Blocked task is removed from the FIFO queue, and placed at the back of queue when it is ready.

Characteristics:

- Preemptive algorithm (of FCFS)
- Response time guarantee: Time taken before a task get CPU is bounded by $(n - 1)q$ where n is number of tasks and q is quantum

- Timer interrupt needed for scheduler to check on quantum expiry
- Choice of time quantum duration matters:
 - Big quantum is higher CPU utilization but longer waiting time
 - Small quantum is bigger overhead but shorter waiting time

Execution Timeline

No other task in queue

Quantum for A expires

Task B and C arrived

A Blocks on I/O

C give up CPU

Mechanism:

- Prioritise task with higher priority value.

Variants:

- Preemptive version
 - Higher priority process can preempt (override) running process with lower priority
- Non-preemptive version
 - Late coming high priority process has to wait for next round of scheduling

Shortcomings:

- Low priority process may starve
- Possible solution:
 - Decrease the priority of currently running process after every time quantum
 - Give the current running process a time quantum and ensure its not considered in next round of scheduling

Tasks	Arrival Time	Priority (1=highest)
A: 8 TUs	Time 0	3
C: 3 TUs	Time 1	1
B: 5 TUs	Time 1	5

Schedule

Mechanism:

- If Priority(A) > Priority(B): run A.
- If Priority(A) == Priority(B): A and B runs in RR

Characteristics:

- Minimizes both response time for IO bound and turnaround time for CPU bound processes.

- 3 Queues: Q2 (highest priority), Q1, Q0
- A single long running job
 - Try to apply the rules and check your understanding

Queues

Q2

Q1

Q0

Time

Lottery Scheduling

Mechanism:

- Give out "Lottery Tickets" to different system resources (CPU Time, I/O devices)
- When a scheduling decision is needed:
 - A lottery ticket is randomly chosen among eligible Tickets to grant the resources
 - In long run, a process holding X% of tickets can win X% of lottery held and use the resource X% of the time

4 Inter-Process Communication

Inter-Process Communication mechanisms is needed for transferring of information between processes.

4.1 Shared-Memory

A shared memory region M is created, which will be attached by the processes P_1 and P_2 , enabling communication.

Advantages:

- Efficient: OS only needs to Create and Attach memory region
- Ease of use: information of any type or size can be written easily in shared memory space

Disadvantage:

- Synchronization between resources is harder
- Implementation is usually harder

Process P1

Process P2

OS is involved in step 1 and 2 only

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/shm.h>

int main() {
    int shmid, *shm;

    // Use shmget syscall to create a shared memory in
    // IPC_PRIVATE: Create a new, private segment that only related
    // processes (e.g. parent/child after fork) can share by passing around
    // this shmid.
    // Create the segment if it doesn't exist, with Unix permissions 0600
    // - owner can read and write, nobody else can.
    shmid = shmget(IPC_PRIVATE, 3 * sizeof(int), IPC_CREAT | 0600 );
    // Exit if Memory cannot be created
    if (shmid == -1) exit(1);

    // Attach memory by initializing the pointer
    // NULL: Let kernel choose free virtual address
    // 0: no extra flag, default read/write access
    shm = (int*) shmat(shmid, NULL, 0);
    // Exit if memory cannot be attached
```

Multi-Level Feedback Queue (MLFQ)

Mechanism:

- If Priority(A) > Priority(B): run A.
- If Priority(A) == Priority(B): A and B runs in RR

Characteristics:

- Minimizes both response time for IO bound and turnaround time for CPU bound processes.

```
if (shm == (int*) -1) exit(1);

// Declare value not ready;
shm[0] = 0;
// Sleep while the value is ready
while(shm[0] == 0) {
    sleep(3);
}

for (int i=0; i<3; i++) {
    printf("Read %d from shared memory. \n", shm[i+1]);
}

// Detach and destroy shared memory region
shmdt((char*) shm);
shmctl(shmid, IPC_RMID, 0);

return 0;
}
```

4.2 Message Passing

Process P_1 prepare messages M and send to P_2 using system calls, where M is stored in kernel memory space. Requires extra properties like Name and Synchronization.

Advantages:

- Portable: can be implemented on diff processing environment
- Easier Synchronization: when sync primitives is used, sender and receiver are implicitly synchronized

Disadvantage:

- Inefficient: Requires OS intervention
- Extra Copying

Process P1

Process P2

- The Msg have to be stored in kernel memory space
- Every send/receive operations need to go through OS (i.e., a system call)

Direct Communication

Sender and Receiver of message explicitly name the other party (ie. unix domain socket). One-to-one communication between processes.

Indirect Communication

Messages are sent / received from message storage known as mailbox or port (i.e. unix message queue). Many-to-Many communication between processes.

Blocking Primitives (synchronous)

Receiver is blocked until message has arrived.

Non-Blocking Primitives (asynchronous)

Receiver either receive the message if available or some indication that message is not ready.

4.2.1 Unix Pipes

In Unix, a process has 3 default communication channels: stdin (scanf), stdout (printf), stderr. Declared using "f".

Pipe functions as:

- Circular bounded byte buffer: Writers wait when buffer is full
- Implicit Synchronization: Readers wait when buffer is empty

Depending on Unix version, pipes may be:

- Half-duplex: unidirectional with one write end and one read end
- Full-duplex: bidirectional with any end for read and write

```
#define READ_END 0
#define WRITE_END 1

int main()
{
    int pipeFd[2], pid, len;
```

```
char buf[100], *str = "Hello There!";

pipe( pipeFd );

if ((pid = fork()) > 0) { /* parent */
    close(pipeFd[READ_END]);
    write(pipeFd[WRITE_END], str, strlen(str)+1);
    close(pipeFd[WRITE_END]);
} else { /* child */
    close(pipeFd[WRITE_END]);
    len = read(pipeFd[READ_END], buf, sizeof(buf));
    printf("Proc %d read: %s\n", pid, buf);
    close(pipeFd[READ_END]);
}
}
```

4.2.2 Unix Signal

Quick form of inter-process communication, sent to a process using an asynchronous notification regarding an event.

- Eg. Kill, Interrupt, Stop, Continue, Memory Error, Arithmetic Error...

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void myOwnHandler( int signo )
{
    if (signo == SIGSEGV){
        printf("Memory access blows up!\n");
        exit(1);
    }
}

int main()
{
    int *ip = NULL;

    if (signal(SIGSEGV, myOwnHandler) == SIG_ERR)
        printf("Failed to register handler\n");

    *ip = 123;

    return 0;
}
```